

FONIX
Voiceln[®]
Game Edition

4.0 SDK
Help
Guide



Table Of Contents

on	1
Additional resources	2
Copyrights and trademarks.....	3
Overview.....	5
Getting started.....	5
Key files	6
Location of files	9
Game tools	11
Adding speech recognition	13
Tutorial	16
Grammar vocabularies	18
Glossary	20
Game API.....	23
FnxVocabInit.....	23
FnxSelectXVocabs	25
FnxVoiceGetResultsGrammar	27
FnxVoiceGetResults	30
FnxVoiceInit.....	34
FnxVoiceTiming	35
FnxVoiceUserInit	37
FnxVoiceRecognize	39
Game tools API	43
FnxVocabBld	43
FnxBuildVoice	46

Installation

VoiceIn Game SDK

1. Create a Fonix directory.
2. Copy the game SDK files to the Fonix directory.
3. Run the batch file:
C:\Fonix\GamerSDK4.1\Examples\data\VoiceCmd\data\vocabBuilderXenon.bat.
4. Compile the Xenon project:
C:\Fonix\GamerSDK4.1\Examples\Xenon\XeVoiceCmd. Execute the program from the Xenon or from the debugger.
5. Plug in an Xenon controller with a communicator headset attached.
6. Push the start button and say a word from the list.
7. **Tutorial** explains how to modify VoiceCmd.cpp to run a multi-user demo (Monsters) and a Japanese demo.

Additional resources

Fonix offers the following services and resources to game developers.

Additional recognizers

- Smaller recognizers (faster but slightly less accurate).
- Larger recognizers (slower but more accurate).

Training and support

- Provided on site or at Fonix's headquarters.
- Provided through e-mail or over the telephone.
- Speech recognition tutorials.
- Help in designing how to use speech in an application.
- Help in integrating speech into an application.
- Multiple language support.

Copyrights and trademarks

IMPORTANT! READ THIS LICENSE AGREEMENT. BY USING THE SOFTWARE, YOU INDICATE YOUR ACCEPTANCE OF THE ENTIRE AGREEMENT.

SOFTWARE LICENSE AGREEMENT

This Agreement is a legal agreement between you and Fonix Corporation ("Fonix"), but may be superseded by a separate license agreement that you and Fonix have signed. The software ("Software") and its media and documentation (collectively, the "Product") are protected by copyright and other intellectual property laws and international treaties. This Product is proprietary to and embodies confidential information and technology of Fonix Corporation. You have no title to, or ownership of, the Product, other than its physical media. Fonix reserves all rights not specifically granted in this Agreement.

USE AND RESTRICTIONS: You may operate the Software on only in a development environment. You may make only one copy of the Software, retaining the Fonix CD or other media for storage and back-up purposes. The Software does not include updates, maintenance, or support, which may be separately available. Any commercialization of the Software and/or Product is strictly prohibited unless a separate written agreed is signed between you and Fonix or an authorized sublicensor. You may not modify, adapt, translate, rent, lease, loan, resell for profit, distribute, network, or create derivative works based on any part of the Software. You may not reverse engineer, decompile, or disassemble the Software.

LIMITED WARRANTY: Fonix warrants that the Product's physical media will be, under normal use, free from defects in material and workmanship for 30 days from the date you purchased it. Fonix's entire liability, and your exclusive remedy under this warranty, will be replacement of the defective media without charge. EXCEPT FOR THIS LIMITED WARRANTY, FONIX MAKES NO WARRANTIES, EXPRESS OR IMPLIED, AND DISCLAIMS ANY IMPLIED WARRANTY OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Fonix does not warrant that the Product is entirely error-free, including but not limited to either technical or editorial errors or omissions. FONIX'S TOTAL LIABILITY FOR ANY CLAIM OR DAMAGE ARISING OUT OF THE USE OF OR INABILITY TO USE THE SOFTWARE SHALL BE LIMITED TO DIRECT DAMAGES, WHICH SHALL NOT EXCEED THE LICENSE FEE YOU PAID. IN NO EVENT SHALL FONIX BE LIABLE FOR LOST PROFITS, LOST DATA, INTERRUPTIONS OF BUSINESS, OR ANY INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF OR INABILITY TO USE THE SOFTWARE, REGARDLESS OF WHETHER FONIX HAS NOTICE OF THE POTENTIAL FOR SUCH LOSS OR DAMAGE. The above limitations may be superseded by law in some jurisdictions.

U.S. GOVERNMENT RESTRICTED RIGHTS: The Software and any accompanying materials are provided with Restricted Rights. Use, duplications, or disclosure by the Government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of The Rights in Technical Data and Computer Software clause at DFARS 252.227-7013 or subparagraphs (c)(1) and (2) of the

Commercial Computer Software-Restricted Rights at 48 CFR 52.227-19, as applicable. Contractor/manufacturer is Fonix Corporation at 9350 South 150 East, Suite 700, Sandy, Utah 84070.

GENERAL: If any term of this Agreement is found void or unenforceable, all other terms shall remain in full force and effect. This Agreement shall be governed by the internal laws of Utah. If you breach any term of this Agreement, Fonix may terminate this license and you must return all copies of the Product to Fonix.

Copyright © 2005 Fonix Corporation, Inc. All rights reserved.

Trademarks

- The Fonix logo and DECtalk are trademarks of Fonix Corporation.
- Compaq is a registered trademark of Compaq Computer Corporation.
- Intel is a trademark of Intel Corporation.
- Linux is a registered trademark of Linus Torvalds.
- Microsoft, Windows, Windows 95, Windows 98, Windows ME, Windows NT, Windows 2000, Windows XP and Windows CE are registered trademarks of Microsoft Corporation.
- Motif is a registered trademark of the Open Software Foundation, Inc.
- Red Hat is a registered trademark of Red Hat Software, Inc.
- SoundBlaster is a registered trademark of Creative Labs, Inc.
- Solaris is a registered trademark of Sun Microsystems, Inc.
- Sparc is a registered trademark of Sun Microsystems, Inc.
- Other product names mentioned in Product documentation may be trademarks and/or registered trademarks of their respective companies.

Getting started

Key topics

- 🔑 **Key files**

Introduces the files and tools used to create a game application with speech recognition.

- 🔑 **Location of files**

Identifies the location of all the Game SDK files, and provides a brief description of each.

- 🔑 **Game tools**

Describes VocabBld.exe (vocabulary build tool) and BuildVoice.exe (voice neural net tool), and the command-line syntax for each.

- 🔑 **Adding speech recognition**

Outlines steps for adding speech recognition to a game application.

- 🔑 **Tutorial**

Presents a complete tutorial using a sample game application called VoiceCmd included with the Game SDK.

Help Guide structure

Overview. Steps and other information for adding Fonix speech recognition to game applications.

Game API. API reference for adding speech recognition.

Game tools API. API reference for creating game tools.

Key files

This topic describes the files used to create speech recognition for games. **Location of files** contains a complete list of files, locations, and brief descriptions of each.

This version of the Fonix Game SDK uses a type of phonetic scoring that provides higher confidence scores than previous versions. To use the earlier scoring mechanism contact Fonix Corporation for the corresponding neural net (.pni) files.

Vocabulary: list of possible recognition words.

Pronunciation dictionary and neural net files

- Language pronunciation dictionaries (.pdc extension) are used by **VocabBld.exe** and **FnxVocabBld** to create a game vocabulary file (.xvocab extension).
- Language neural net files (.psi extension) are used by **VocabBld.exe** and **BuildVoice.exe**, FnxAdaptVocab and **FnxBuildVoice** to create game specific vocabulary (.xvocab extension), neural net (.vnn extension), and user files (.usr extension). See **Game files** below.
- Fonix pronunciation dictionaries (.pdc) and neural net files (.psi) are not used in games. They are only used to create the vocabularies and voice neural nets used in games.
- See **Location of files** for the location of dictionary and neural net files.
- Available languages: U.S. English, U.K. English, Spanish, French, German, Japanese, Italian and Korean.

Word list file

- **VocabBld.exe** and **BuildVoice.exe** use word lists (.txt extension) to create the **game files** described below. A game can use more than one vocabulary file at a time. For example, a game may have a main menu, several submenus, and several sets of commands, one or more of which may be active during the game.
- Game developers provide word lists as ASCII or Unicode files. Each word is preceded by a word ID number, which the ASR engine returns when the word is recognized.
- Word list format:

```
[ID] [tab] [word or phrase]\n
```

ID: positive integer from 0 to 65535 (returned for each recognized word).
- Lines not beginning with a digit are ignored.
- Example:

Main menu (this line is ignored)

1001 Villager

1002 Barkeep

Submenu (this line is ignored)

1003 Bar

1004 Cabin

Commands (this line is ignored)

1005 Open the door

1006 Cross the street

🔴 Tips for creating a good word list:

1. Keep track of each word ID and the word or phrase it represents. The text is not returned.
2. Spell out words. Example: "m sixteen" instead of "M-16" and "five dollars" instead of "\$5."
3. Do not use punctuation. Exception: the apostrophe in contractions (don't, can't, wouldn't, etc.).
4. Word lists are not case sensitive.
5. Use Unicode or the ISO-8859-1 character set (ANSI Windows) for modified characters, such as those used in foreign languages. Example characters: á, ë and î.
6. Enter Japanese characters using Romanji (ASCII text), or Unicode using Katakana or Hiragana.
7. Spell difficult names the way they sound. For example, if the game name is Xphyr (pronounced ziffer), spell it "ziffer" in the word list.

Game files

🔴 Vocabulary file (.xvocab extension)

1. Word lists are converted into vocabulary files by **VocabBld.exe** (vocabulary build tool) or by calling **FnvVocabBld** (game API).
2. Load vocabulary files only once (file immediately available to all users).
3. Initialize vocabularies by calling **FnvVocabInit** (vocabulary loaded directly into the game).
4. Select vocabularies by calling **FnvSelectXVocabs**. Knowing when to select a vocabulary depends on the game.

5. Games may simultaneously use multiple vocabulary files. Multiple vocabularies are treated as one vocabulary for recognition purposes.

🔸 Voice neural net file (.vnn extension)

6. Voice neural net files are created by **BuildVoice.exe** (voice neural net tool) or by calling **FnxBuildVoice** (game API).
7. Load a voice neural net only once (file immediately available to all users).
8. Initialize a voice neural net by calling **FnxVoiceInit** (neural net loaded directly into the game). Unless switching between multiple languages, a game typically uses only one neural net file.
9. Voice neural net (.vnn) files contain information from the vocabulary files. If a vocabulary file changes, the corresponding voice neural net must also be rebuilt.

🔸 Voice user file (.usr)

1. Voice user files are created by **BuildVoice.exe** (voice user tool) or by calling **FnxBuildVoice** (game API).
2. Initialize a user file by calling **FnxVoiceUserInit** (user file loaded directly into the game).

NOTE: **FnxVoiceInit** must be called before **FnxVoiceUserInit**.

3. A copy of the voice user file must be initialized for every person playing a particular game.
4. User (.usr) files contain information from the vocabulary files. If a vocabulary file changes, the corresponding user file must also be rebuilt.

Related topic

Game tools; Adding speech recognition

Location of files

Help guide

GamerSDK4.1\Docs	
File	Description
GameSDK41.pdf	The Fonix Game SDK Help guide in .pdf file format.
GameSDK41.chm	The Fonix Game SDK Help guide in html format.

Source code and documentation

Fonix\GamerSDK4.1\Examples\Xenon\XeVoiceCmd	
File	Description
XeVoiceCmd.cpp	Source code for English and Japanese single-user.
XeVoiceCmd.vcproj	Project file for demo.
ReadMe.txt	Documentation on how to build the XeVoiceCmd demo.
Fonix\GamerSDK4.1\include	
File	Description
VoiceCmds.h	Header file for VoiceCmds.lib.
Fonix\GamerSDK4.1\lib\Xenon	
File	Description
VoiceCmds.lib	Speech recognition library.

Language dictionaries and neural net files

Fonix\GamerSDK4.1\Languages\French	
File	Description
French.pdc	French pronunciation dictionary.
French.psi	Packed neural net file.
Fonix\GamerSDK4.1\Languages\German	
File	Description
German.pdc	German pronunciation dictionary.
German.psi	Packed neural net file.
Fonix\GamerSDK4.1\Languages\Italian	
File	Description
Italian.pdc	Italian pronunciation dictionary.
Italian.psi	Packed neural net file.
Fonix\GamerSDK4.1\Languages\Japanese	
File	Description
Japanese.pdc	Japanese pronunciation dictionary.
Japanese.psi	Packed neural net file.
Fonix\GamerSDK4.1\Languages\Korean	
File	Description
Korean.pdc	Korean pronunciation dictionary.
Korean.psi	Packed neural net file.
Fonix\GamerSDK4.1\Languages\Spanish	

File	Description
Spanish.pdc	Spanish pronunciation dictionary.
Spanish.psi	Packed neural net file.
Fonix\GamerSDK4.1\Languages\UKEnglish	
File	Description
UKEnglish.pdc	U.K. English pronunciation dictionary.
UKEnglish.psi	Packed neural net file.
Fonix\GamerSDK4.1\Languages\USEnglish	
File	Description
USEnglish.pdc	U.S. English pronunciation dictionary.
USEnglish.psi	Packed neural net file.

Game SDK tools

GamerSDK4.1\tools\Win32\bin	
File	Description
VocabBld.exe	Tool to build .xvocab files.
BuildVoice.exe	Tool to build .vnn and .usr files.
GamerSDK4.1\tools\Win32\VocabBld	
File	Description
VocabBld.c	Source code for VocabBld.exe
VocabBld.vcproj	Project file for VocabBld.exe
GamerSDK4.1\tools\Win32\BuildVoice	
File	Description
BuildVoice.c	Source code for BuildVoice.exe
BuildVoice.vcproj	Project file for BuildVoice.exe
GamerSDK4.1\tools\Win32\include	
File	Description
VocabLib.h	Header file for VocabLib.lib
GamerSDK4.1\tools\Win32\lib	
File	Description
VocabLib.lib	Win32 static library used by VocabBld.c and BuildVoice.c

Related topic

Game tools

Game tools

The command-line tools described below (VocabBld.exe and BuildVoice.exe) create files for doing speech recognition on the Microsoft game platform.

Game developers can include the same functionality in their own development tools:

1. Include VocabLib.h.
2. Link VocabLib.lib to the development project.
3. Call functions from the game tools API (**FnxVocabBld** and **FnxBuildVoice**).
4. Review the source code provided for both tools (VocabBld.c and BuildVoice.c).
5. Review **Key files** for more information on game files.

VocabBld.exe (vocabulary build tool)

Creates a vocabulary file (.xvocab extension) from a word list. Command-line syntax:

VocabBld.exe <word list> <dictionary> <neural net> <output vocabulary file> [-n <nBest>] [-prune <pruning level>]

- **<word list>** Text file containing a word list (.txt extension). See **Word list file**.
- **<dictionary>** Language pronunciation dictionary (.pdc extension). See **Pronunciation dictionary** and neural net files.
- **<neural net>** Language packed neural net (.psi extension). See Pronunciation dictionary and **neural net files**.
- **<output vocabulary file>** Binary vocabulary file (.xvocab extension) used for recognition. See **Game files**.
- **[-n <nBest>]** (optional) Number of top answers to find from the vocabulary (default is 1).
- **[-prune <pruning level>]** (optional) This setting effects a trade-off between speed and recognition accuracy. Value range is 0 to 100: 0 is fastest; 50 is the default (fast and accurate); 100 is the most accurate.

NOTE: See **Comments** under **FnxVocabBld** for more details.

BuildVoice.exe (voice neural net and user tool)

Creates neural net (.vnn extension) and user (.usr extension) files. Run tool only after one or more vocabulary files have been created. Command-line syntax:

```
BuildVoice.exe <output neural net file> <output user file> <input  
neural net> <vocabulary file> [vocabulary file2...] [-s  
<MaxSimultaneousVocabs>] [-p MaxTotalSearchPaths] [-n MaxTotalNodes]
```

- **<output neural net>** Name of the output voice neural net file (.vnn).
- **<output user file>** Name of the output voice user file (.usr).
- **<input neural net>** Language neural net (.psi). See Pronunciation dictionary and [neural net files](#).
- **<vocabulary file>** Name of a vocabulary file (.xvocab). Add multiple vocabulary files in sequence.
- **<MaxSimultaneousVocabs>** (optional) Maximum number of vocabularies used simultaneously.
- **<MaxTotalSearchPaths>** (optional) Maximum number of search paths.
- **<MaxTotalSearchNodes>** (optional) Maximum number of search nodes.
- **<MaxTotalNodes>** (optional) Maximum number of total nodes.

NOTE: The default settings for maximum search paths and maximum search nodes are large enough to handle the largest number of vocabulary files to be used simultaneously. To save memory, add up these values for the largest set of vocabularies used simultaneously. See **Comments** under [FnxBuildVoice](#) for more details.

Related topics

[Key files](#); [FnxVocabBld](#); [FnxBuildVoice](#)

Adding speech recognition

This topic covers the steps for adding speech recognition to games.

Steps

1. Create a **word list file**.
2. Create one or more vocabulary files. See **VocabBld.exe** and/or **FnxVocabBld**.
3. Create voice neural net and voice user files. See **BuildVoice.exe** and/or **FnxBuildVoice**.
4. Copy all **game files** (.xvocab, .vnn, .usr) to a device or reference directory.
5. Include **VoiceCmds.h** in the project.
6. Link **VoiceCmds.lib** to the project.
7. Decide when to initiate speech recognition. It can start in response to a button being pushed on the game controller, immediately after a question is asked, or at another juncture as determined by the game's developer. Turn on speech recognition only when it is necessary. Leaving the recognizer on when it is not necessary can result in false recognition. It should be clear to the user when a button should be pushed, or when simply speaking is enough to start speech recognition.
8. Load one or more vocabulary files into memory. Initialize each vocabulary file by calling **FnxVocabInit**.

```
// code snippet
FnxVocabPtr *ppFnxVocabs; // void pointer defined in VoiceCmds.h
int iNumVocabs = 3; // multiple vocabulary files

ppFnxVocabs = (FnxVocabPtr *)calloc( iNumVocabs,
sizeof(FnxVocabPtr) );
for( i = 0; i < iNumVocabs; i++ )
{
    ppFnxVocabs[i] = ReadBinaryFile( pwsFnxVocabFiles[i], &iSize
    );
    err = FnxVocabInit( ppFnxVocabs[i] );
}
}
```

9. Load a voice neural net file into memory. Initialize it by calling **FnxVoiceInit**.

```
// code snippet
int err;
FnxVoicePtr pFnxVoice = (FnxVoicePtr) ReadBinaryFile(
wsFnxVoiceFile, &iSize );
err = FnxVoiceInit( pFnxVoice );
```

10. Load up to four voice user files into memory. Initialize each voice user file by calling **FnxVoiceUserInit**.

```
// code snippet
for( i = 0; i < 4; i++ )
{
    if( ppCommunicator[i] ) // See if we have controller and
        headset in port i
    {
        ppCommunicator[i]->pFnxVoiceUser =
            (FnxVoiceUserPtr)ReadBinaryFile( wsFnxVoiceUserFile,
                &iSize );
        err = FnxVoiceUserInit( pFnxVoice, ppCommunicator[i]-
            >pFnxVoiceUser );
    }
}
```

11. Select one or more vocabularies by calling **FnxSelectXVocabs**. Knowing when to select a vocabulary depends on the game. If more than one vocabulary is selected at a particular point in the game, the recognizer returns the best **word match ID** from all selected vocabularies.

```
// code snippet
FnxVoiceUserPtr pFnxVoiceUser; // void pointer defined in
VoiceCmds.h

// select three vocabularies -- the order is not important
ppChosenVocabs[0] = ppFnxVocabs[3];
ppChosenVocabs[1] = ppFnxVocabs[7];
ppChosenVocabs[2] = ppFnxVocabs[1];
```

```
iNumChosenVocabs = 3;
err = FnxSelectXVocabs( pFnxVoiceUser, ppChosenVocabs,
    iNumChosenVocabs );
```

12. Begin recognition (call **FnxVoiceRecognize**) when the push-to-talk button is pushed, after a question is asked, or when the game knows in some other way to expect speech.

```
// SPEECH_DETECTED, END_OF_SPEECH, and RESULTS_AVAILABLE are
flags
// passed back through puiResult.
err = FnxVoiceRecognize( pFnxVoice, pFnxVoiceUser,
    pwSpeechSamples, iNumSamples, &puiResult );
```

13. Check for end-of-speech. If puiResult & END_OF_SPEECH is true, stop sending speech data. Continue calling **FnxVoiceRecognize**, setting pwSpeechSamples to NULL and iNumSamples to 0.
14. Check for available results. If puiResult & RESULTS_AVAILABLE is true, speech recognition is done.
15. Get speech results by calling **FnxVoiceGetResults**.

```
DWORD *pdwWordID;
FLOAT *pfConfidence;
DWORD n;
```

```
err = FnxVoiceGetResults( pFnxVoice, pFnxVoiceUser, &pdwWordID,  
&pfConfidence, &nBest );
```

16. Evaluate results. The nBest word IDs are in pdwWordID[0..nBest-1]. A confidence score for each word ID is in pfConfidence[0..nBest-1]. The word is incorrect if the confidence is negative.

Related topic

[Game files](#); **[Tutorial](#)**

Tutorial

XeVoiceCmd

This topic covers compiling and running XeVoiceCmd, a single user and single vocabulary voice recognition program.

File location: Fonix\GamerSDK4.1\Examples\Xenon\XeVoiceCmd\data (add this path to **VocabBld.exe** and **BuildVoice.exe** below).

- Word list text file: Tasty.txt.
- Create a vocabulary file by converting Tasty.txt to Tasty.xvocab.
 - Open a console window.
 - Type: **VocabBld.exe Tasty.txt USEnglish.pdc USEnglish.psi Tasty.xvocab -n 10**
 - -n 10 is not required. Most games use the default value (1). See **FnxVocabBld**.
- Create voice neural net (.vnn) and voice user (.usr) files.
 - Open a console window.
 - Type: **BuildVoice.exe Tasty.vnn Tasty usr USEnglish.psi Tasty.xvocab**
 - Copy Tasty.vnn, Tasty usr, and Tasty.xvocab to \Xenon\XeVoiceCmd\Data on the game platform (sample code is coded to look in this location).
 - You will need to copy Tasty.txt to the data location in order to display words on the screen, but keep in mind that most games do not necessarily need the text file once the vocabulary files are built.
- Compile XeVoiceCmd
 - Open **XeVoiceCmd.vcproj** (project file).
 - If not already linked, link **VoiceCmds.lib** to the project.
 - Run the program.
 - 70 words are displayed on the game screen.
 - Press and release the start button on the game controller.
 - Speak a word from the list.
 - The top 10 words appear in the upper-left corner of the display. The top choice is in yellow.
 - Next to the top 10 words appears a confidence score and the word ID.

Related topic

Game tools

Grammar vocabularies

Grammar vocabularies can extract a series of words from a speech utterance, according to a set of rules defined by the grammar. Grammar rules may dictate sequence, repetition, groupings, optional items, and Boolean expressions. Allowances can be made for the presence of noise and silence before and after words. Recognition words can be mapped to a value or word.

Grammar vocabularies are helpful in situations when specific word combinations are important for recognition. For example, answering questions such as "who," "what," or "where" in a game. Grammars allow for the inclusion of unneeded words within word combinations to be recognized.

Grammar syntax definitions

Grammar syntax definitions are divided into six categories: variables, repetition, grouping/optional items, Boolean, output directives, and key words. These syntax definitions are used in creating a grammar.

Variables

Used to name sets of recognition words. The number of variables is not limited.

- ☛ \$ identifies a variable. A space does not separate the \$ and the variable.

Repetition/Optional items

Used to create optional variables or the unlimited repetition of the same variable.

- ☛ [] encloses optional items. The enclosed items are not required for recognition to occur. If they occur, they can occur only once.
- ☛ {} encloses items that may or may not be repeated more than one time.
- ☛ <> encloses items that are required once but may be repeated multiple times. The enclosed items must occur at least once.

Grouping

Parentheses () enclose simple groupings. As in math equations, parentheses may be used to group items in grammars and word sets to add clarity and readability. They are not required but may help in writing complicated grammars.

Boolean

Boolean expressions are used to write how recognition words and variables relate to other words and variables inside variables and grammars.

Symbol	Meaning
	OR

space or blank	AND
----------------	-----

Output directives

These symbols are used to map a recognition word to an output word. This is helpful for recognizing numbers and displaying numbers as a series of digits. These symbols can also be used to represent a word with an abbreviation or with another word.

Symbol	Meaning
%	output word ID
%%	no output

Examples

Output directive	Result
two%2	2 (word ID)
two%%	no output

Key words

The key words below that may not be redefined by the user.

Name	Recognition Word	Input
\$Any		noise and out of vocabulary words
\$grammar		required variable name of the grammar

Related topic

[FnxVoiceGetResultsGrammar](#)

Glossary

.pdc

ASR pronunciation file containing a fixed number of words with phonemic representations (pronunciations).

.pni

Compressed (packed neural net file. No longer used except for backward compatibility. See .psi below.

.psi

Compressed (packed) neural net file containing statistical (recognition) information for **phonetic scoring**.

.usr

Voice user file.

.vnn

Voice neural net file.

.xvocab

Vocabulary file.

Audio collection

Synonymous with speech recording.

Command and control

Industry term describing a system that responds to input commands to determine what the system will do.

Dictionary

Contains the number of words with corresponding pronunciations that can be looked up and matched to a vocabulary word by the speech recognizer.

Grammar

Determines how a vocabulary can be put together to achieve higher recognition accuracy.

nBest

Maximum number of recognition results to return. Results are listed from highest to lowest according to the word's recognition score.

Neural network (neural net)

A multi-tiered data processing model that mimics (in an scaled down fashion) the logic that takes place in the human brain. A neural net must be trained for each language.

Out-of-vocabulary word

Any word or phrase not in the set of recognition words.

Pruned neural network

A neural net modified to increase processing speed at some cost to rejecting out-of-vocabulary words.

Rejection

Disregarding out-of-vocabulary words and phrases.

Speaker-independent recognition

A speech-recognition system that requires no speech training.

Speech detector

Function in the ASR engine that determines where speech begins and ends during data collection.

Speech recognition

Technology that enables a computer to recognize speech. Fonix ASR is speaker-independent and supports continuous, connected speech. It is ideally suited for **command-and-control** applications.

Speech utterance

A single word or series of words. Groups of multiple words contain no long pauses.

Text-to-speech (TTS)

Technology that enables a computer to convert (digitize) input text and speak it.

Transcription (pronunciation)

The phoneme representation of a word or phrase.

Vocabulary (recognition words)

Words and/or phrases assigned to a node for recognition by the ASR engine. Recognition words should not be confused with a recognition dictionary. A node's vocabulary (recognition words) is set with a create node function.

Word list

Fonix VoiceIn Game Edition

Text file of words that make up a game's vocabulary.

FnxVocabInit

Description

Initializes a vocabulary file (.xvocab extension) for recognition.

Syntax

HRESULT **FnxVocabInit**(**FnxVocabPtr** *pFnxVocab*)

Parameters

<i>pFnxVocab</i>	FnxVocabPtr	Pointer to a vocabulary file. The pointer is returned by ReadBinaryFile or similar function. See Example code below.
------------------	--------------------	---

Returns

FNX_ERR_NONE	Successful.
FNX_NULL_POINTER	<i>pFnxVocab</i> is NULL.
FNX_WRONG_TYPE	<i>pFnxVocab</i> does not point to a valid vocabulary file.
FNX_INCOMPLETE	The footer on the <i>pFnxVocab</i> block is incorrect (memory overrun).
FNX_VERSION_MISMATCH	<i>pFnxVocab</i> is not the correct version (e.g., built by obsolete code).
FNX_NOT_INITIALIZED	<i>pFnxVocab</i> has not been initialized.
FNX_UNALIGNED_BLOCK	<i>pFnxVocab</i> does not begin at an even 4-byte memory address. Rebuild the vocabulary file.

Comments

All users share the same vocabulary file. Do not initialize more than once.

Example code

```
// code snippet inside the initialization routine of the main
application
int iNumVocabs = 3;
FnxVocabPtr ppFnxVocabs[3];

// Use some function to read the binary files into memory.
ppFnxVocabs[0] = (FnxVocabPtr) ReadBinaryFile( "<vocab1>.xvocab",
&iSize );
ppFnxVocabs[1] = (FnxVocabPtr) ReadBinaryFile( "<vocab2>.xvocab",
&iSize );
ppFnxVocabs[2] = (FnxVocabPtr) ReadBinaryFile( "<vocab3>.xvocab",
&iSize );
```

```
if( pFnxVocabs[0] == NULL || pFnxVocabs[1] == NULL || pFnxVocabs[2]
== NULL )
    return -1; // Error: Probably couldn't open one of the files.

if( FnxVocabInit(ppFnxVocabs[0]) != 0
    || FnxVocabInit(ppFnxVocabs[1]) != 0
    || FnxVocabInit(ppFnxVocabs[2]) != 0 )
{
    return -2; // Error in initialization. Vocabulary file may not be
    valid.
}
```

Related topic

[Adding speech recognition](#)

FnxSelectXVocabs

Description

Selects one or more vocabulary files (.xvocab extension) for recognition.

Syntax

HRESULT **FnxSelectXVocabs**(**FnxVoiceUserPtr** *pFnxVoiceUser*,
FnxVocabPtr **ppFnxVocabs*, **int** *iNumVocabs*)

Parameters

<i>pFnxVoiceUser</i>	FnxVoiceUserPtr	Pointer to a voice user file initialized by FnxVoiceUserInit .
<i>ppFnxVocabs</i>	FnxVocabPtr	Array of pointers to one or more vocabulary files.
<i>iNumVocabs</i>	int	Number of vocabularies in <i>ppFnxVocabs</i> .

Returns

FNX_TOO_MANY_VOCABS	<i>iNumVocabs</i> exceeds the maximum number of simultaneous vocabularies. See Comments below.
FNX_NODE_OVERFLOW	Number of search nodes used by the selected vocabularies exceeds the maximum number set by BuildVoice.exe or FnxBuildVoice .
FNX_NULL_POINTER	<i>pFnxVoiceUser</i> or one of the elements in <i>ppFnxVocabs</i> is NULL.
FNX_WRONG_TYPE	<i>pFnxVoiceUser</i> does not point to a valid voice user file, or one of the elements in <i>ppFnxVocabs</i> does not point to a valid vocabulary file.
FNX_NOT_INITIALIZED	<i>pFnxVoiceUser</i> or one of elements in <i>ppFnxVocabs</i> is not initialized.
FNX_TAG_MISMATCH	<i>pFnxVoiceUser</i> and one or more of the elements in <i>ppFnxVocabs</i> were not created at the same time (creation not based on the same files). Rerun BuildVoice.exe or FnxBuildVoice .

Comments

- Call **FnxVocabInit**, **FnxVoiceInit**, and **FnxVoiceUserInit** before calling **FnxSelectXVocabs**.
- Oftentimes only a single vocabulary is needed (recognition words). Under certain conditions, multiple vocabularies are useful. For example, a game may need access to one set of phrases at any point in the game (e.g., "pause

game" and "save game"). Other activities requiring different vocabularies may change over the course of the game. **FnXSelectXVocabs** can specify multiple vocabularies. The recognizer returns the best word or phrase match from all initialized vocabularies.

- 🔴 **VocabBld.exe** and **BuildVoice.exe** specify the maximum number of simultaneous vocabularies. *iNumVocabs* must not exceed this maximum.

Example code

```
int iNumVocabs = 3;
FnXVoiceUserPtr pFnXVoiceUser; //...see FnXVoiceUserInit example
code
FnXVocabPtr ppFnXVocabs[3]; //...see FnXVocabInit example code

// use <vocab1.xvocab> and <vocab2>.xvocab
FnXVocabPtr ppSelections[2];
int iNumSelections = 2;

ppSelections[0] = ppFnXVocabs[0];
ppSelections[1] = ppFnXVocabs[1];

if( FnXSelectXVocabs( pFnXVoiceUser, ppSelections, iNumSelections )
!= 0)
    return -1; // error
else
    // Ready to do recognition with pFnXVoiceUser.
```

Related topic

Adding speech recognition

FnxVoiceGetResultsGrammar

Description

Retrieves the word IDs and confidence levels of the best words found during speech recognition, using a grammar for the vocabulary. The grammar is loaded and initialized the same as any vocabulary. See **Example grammar** below.

Syntax

HRESULT **FnxVoiceGetResultsGrammar**(**FnxVoicePtr** *pFnxVoice*, **FnxVoiceUserPtr** *pFnxVoiceUser*, **DWORD** ****pppdwWordIdList*, **DWORD** ***ppwdwNumWords*, **FLOAT** ***ppfConfidence*, **int** **piNBest*)

Parameters

<i>pFnxVoice</i>	FnxVoicePtr	Pointer to voice neural net file initialized by FnxVoiceInit ..
<i>pFnxVoiceUser</i>	FnxVoiceUserPtr	Pointer to a voice user file initialized by FnxVoiceUserInit . Results are returned for this user file or headset..
<i>pppdwWordIdList</i>	DWORD	Pointer to an array of ID lists containing word IDs found during recognition. The lists correspond to the grammar initialized with FnxVocabInit . Set to NULL if not needed (e.g., discard recognition results and reset recognition for a new utterance).
<i>ppwdwNumWords</i>	DWORD	Array of number of words corresponding to each of the word ID lists contained in <i>pppdwWordIdList</i> .
<i>ppfConfidence</i>	FLOAT	Array of confidence levels corresponding to the word IDs in <i>pppdwWordIdList</i> . Set to NULL if not needed.
<i>piNBest</i>	int	Array of the number of nBest results returned in <i>pppdwWordIdList</i>. 0 means nothing was recognized. The actual number returned typically represents the largest of the nBest parameters used by the current vocabulary (see VocabBld.exe or

		FnxVocabBld). It may be less if fewer word candidates were found. The first word is usually the correct one. The rest of the words in the list represent the most confident mismatches. Most games should use the default nBest = 1, especially since nBest > 1 slows down processing and increases memory requirements
--	--	---

Returns

FNX_ERR_NONE	Successful.
FNX_NULL_POINTER	<i>pFnxVoice</i> or <i>pFnxVoiceUser</i> is NULL.
FNX_WRONG_TYPE	<i>pFnxVoice</i> was not point to a valid voice neural net file, or <i>pFnxVoiceUser</i> does not point to a valid voice user file.
FNX_NOT_INITIALIZED	<i>pFnxVoice</i> or <i>pFnxVoiceUser</i> were not initialized by FnxVoiceInit or FnxVoiceUserInit .

Comments

- Call **FnxVoiceGetResultsGrammar** after RESULTS_AVAILABLE is returned from one or more calls to **FnxVoiceRecognize**. **FnxVoiceGetResultsGrammar** retrieves the word ID and the confidence level of the words or phrases in the currently selected vocabulary or set of vocabularies that best match the wave data passed into **FnxVoiceRecognize**. Multiple vocabularies are used for recognition as if they were one vocabulary.
- If not NULL, *pppdwWordIdList* and *ppfConfidence* receive pointers to permanent arrays. Do not free or modify them. They are overwritten during the next call to **FnxVoiceGetResultsGrammar**.
- It is possible to call **FnxVoiceGetResultsGrammar** before RESULTS_AVAILABLE is returned from **FnxVoiceRecognize**. For example, if a game uses the same button to start and stop recognition, **FnxVoiceGetResults** can be called to reset *pFnxVoiceUser* (make it ready for the next round of recognition). A game might also need an answer within a certain amount of time. As long as SPEECH_DETECTED is returned by **FnxVoiceRecognize** by the time the limit is up, the game might call **FnxVoiceGetResultsGrammar** to find out which word most closely matches what was said so far.

- Recognition on the same utterance is not possible after calling **FnxVoiceGetResultsGrammar**. The internal state of *pFnxVoiceUser* is changed so it is reset on the next call to **FnxVoiceRecognize**.
- Negative confidence scores mean the correct word was probably not found. For example, if "yes" and "no" are the only choices, and the user says "maybe," "yes" might be returned with a negative confidence value. This indicates that the best word is still not correct because "maybe" is not in the vocabulary.
- A recognized word is probably correct if the confidence level is high (above 20). Small positive values between 0 and 10 may or may not indicate **out-of-vocabulary** words. The game developer must determine if the game should err on the side of accepting false positives, or possibly reject correct words. Although not completely reliable, a common approach is to reject words with negative confidence scores and accept words with positive scores. If in critical situations the confidence score is too low, the user might be prompted to repeat an answer.

Example grammar

The number following the percent sign represents a word ID. See **Related topic** for a full explanation.

```
"$who = (team%27 | able%28 | bravo%29);"
```

```
"$what = (\\"hold fire\\"%1 | \\"fire at will\\"%2 | \\"cover area\\"%3 |
deploy%4 | ambush%6 | \\"run to\\"%7 | \\"lead to\\"%8 | \\"attack to\\"%9
| \\"stealth to\\"%10 | regroup%11 | follow%12 | \\"hold
position\\"%13);"
```

```
"$where = (frag%14 | smoke%15 | bang%16 | crosshairs%17 | charlie%18
| delta%19 | echo%20 | foxtrot%21 | juliet%22 | romeo%23 |
whiskey%24 | ex ray%25 | zulu%26);"
```

```
"$grammar = $who $what [$where];"
```

Related topic

Grammar vocabularies

FnxVoiceGetResults

Description

Retrieves the word IDs and confidence levels of the best matched words during speech recognition.

Syntax

```
HRESULT FnxVoiceGetResults( FnxVoicePtr pFnxVoice, FnxVoiceUserPtr
pFnxVoiceUser, DWORD **ppdwWordId, FLOAT **ppfConfidence, int *piNBest
)
```

Parameters

<i>pFnxVoice</i>	FnxVoicePtr	Pointer to voice neural net file initialized by FnxVoiceInit .
<i>pFnxVoiceUser</i>	FnxVoiceUserPtr	Pointer to a voice user file initialized by FnxVoiceUserInit . Results are returned for this user file or headset.
<i>ppdwWordId</i>	DWORD (optional)	Array of word IDs found during recognition. Set to NULL if not needed (e.g., discard recognition results and reset recognition for a new utterance).
<i>ppfConfidence</i>	FLOAT (optional)	Array of confidence levels corresponding to the word IDs in <i>ppdwWordId</i> . Set to NULL if not needed.
<i>piNBest</i>	int	Number of word IDs returned in <i>ppdwWordId</i>. 0 indicates that no words were recognized (e.g., vocabulary not selected, or insufficient number of speech samples were passed). The actual number returned typically represents the largest of the nBest parameters used by the current vocabulary (see VocabBld.exe or FnxVocabBld). It may be less if fewer word candidates were found. The first word is usually the correct one. The rest of the words in the list represent

		the most confident mismatches. Most games should use the default nBest = 1, especially since nBest > 1 slows down processing and increases memory requirements.
--	--	---

Returns

FNX_ERR_NONE	Successful.
FNX_NULL_POINTER	<i>pFnxVoice</i> or <i>pFnxVoiceUser</i> is NULL.
FNX_WRONG_TYPE	<i>pFnxVoice</i> was not point to a valid voice neural net file, or <i>pFnxVoiceUser</i> does not point to a valid voice user file.
FNX_NOT_INITIALIZED	<i>pFnxVoice</i> or <i>pFnxVoiceUser</i> were not initialized by FnxVoiceInit or FnxVoiceUserInit .

Comments

- Call **FnxVoiceGetResults** after RESULTS_AVAILABLE is returned from one or more calls to **FnxVoiceRecognize**. **FnxVoiceGetResults** retrieves the word ID and the confidence level of the words or phrases in the currently selected vocabulary or set of vocabularies that best match the wave data passed into **FnxVoiceRecognize**. Multiple vocabularies are used for recognition as if they were one vocabulary.
- If not NULL, *ppdwWordId* and *ppfConfidence* receive pointers to permanent arrays. Do not free or modify them. They are overwritten during the next call to **FnxVoiceGetResults**.
- It is possible to call **FnxVoiceGetResults** before RESULTS_AVAILABLE is returned from **FnxVoiceRecognize**. For example, if a game uses the same button to start and stop recognition, **FnxVoiceGetResults** can be called to reset *pFnxVoiceUser* (make it ready for the next round of recognition). A game might also need an answer within a certain amount of time. As long as SPEECH_DETECTED is returned by **FnxVoiceRecognize** by the time the limit is up, the game might call **FnxVoiceGetResults** to find out which word most closely matches what was said so far.
- Recognition on the same utterance is not possible after calling **FnxVoiceGetResults**. The internal state of *pFnxVoiceUser* is changed so it is reset on the next call to **FnxVoiceRecognize**.
- Negative confidence scores mean the correct word was probably not found. For example, if "yes" and "no" are the only choices, and the user says "maybe," "yes" might be returned with a negative confidence value. This indicates that the best word is still not correct because "maybe" is not in the vocabulary.
- A recognized word is probably correct if the confidence level is high (above 20). Small positive values between 0 and 10 may or may not indicate **out-of-vocabulary** words. The game developer must determine if the game should err on the side of accepting false positives, or possibly reject correct

words. Although not completely reliable, a common approach is to reject words with negative confidence scores and accept words with positive scores. If in critical situations the confidence score is too low, the user might be prompted to repeat an answer.

Example code

```
// code snippet inside the processing loop for a headset communicator.

FnxVoicePtr pFnxVoice; // Global Fonix voice structure.
FnxVoiceUserPtr pFnxVoiceUser; // Fonix voice user structure for the
selected user or headset.
HRESULT err;

// example of the state information a game keeps for each user
enum {VOICE_IDLE, // push-to-talk button has not been pushed.
VOICE_LISTENING, // recognition is currently being done on mic input
VOICE_FINISHING, // END_OF_SPEECH detected, but not necessarily done
VOICE_DONE}; // RESULTS_AVAILABLE returned and results retrieved.

// application can return to VOICE_IDLE once the results are used
DWORD uiResult;
int iState;

// one for each available microphone packet
if( iState == VOICE_LISTENING || iState == VOICE_FINISHING )
{
    // submit the copied data to the speech recognizer
    if (iState == VOICE_FINISHING) // END_OF_SPEECH returned
    {
        // provide time to finish doing recognition without sending
        more data
        err = FnxVoiceRecognize( pFnxVoice, pVoiceUser, NULL, 0,
            &uiResult );
    }
    else // END_OF_SPEECH not returned -- continue passing real
    microphone data
    {
        // send in and process wave data (2 bytes per sample)
        err = FnxVoiceRecognize( pFnxVoice, pVoiceUser,
            pwMicrophoneData, iMicrophonePacketSizeInBytes / 2, &uiResult
        );

        if( uiResult & END_OF_SPEECH )
        {
            iState = VOICE_FINISHING;
        }
    }
}

// check for recognition results
if( uiResult & RESULTS_AVAILABLE )
{
    int iNBest;
    DWORD *pdwWordID, iBestWordID;
    FLOAT *pfConfidence, iBestConfidence;
```

```
// Processing has completed on the speech, so get the results
err = FnxVoiceGetResults(pFnxVoice, pVoiceUser, &pdwWordID,
&pfConfidence, &iNBest);
if( iNBest > 0 && !err )
{
    // top answer is in the first array element
    iBestWordID = pdwTempWordID[0];
    fBestConfidence = pfConfidence[0];
}
iState = VOICE_DONE;
}
```

FnxVoiceInit

Description

Initializes a voice neural net file (.vnn).

Syntax

HRESULT **FnxVoiceInit**(**FnxVoicePtr** *pFnxVoice*)

Parameters

<i>pFnxVoice</i>	FnxVoicePtr	Pointer to a voice neural net file. The pointer is returned by ReadBinaryFile or similar function. See Example code below.
------------------	--------------------	---

Returns

FNX_ERR_NONE	Successful.
FNX_NULL_POINTER	<i>pFnxVoice</i> is NULL.
FNX_WRONG_TYPE	<i>pFnxVoice</i> does not point to a valid neural net file.
FNX_INCOMPLETE	The footer on the <i>pFnxVoice</i> block is incorrect (memory overrun).
FNX_VERSION_MISMATCH	<i>pFnxVoice</i> is not the correct version (e.g., built by obsolete code).
FNX_UNALIGNED_BLOCK	<i>pFnxVoice</i> does not begin at an even 4-byte memory address. Rebuild vocabulary and voice neural net files.

Comments

All user files and vocabularies use the same voice neural net file. Do not initialize more than once.

Example code

```
FnxVoicePtr pFnxVoice;
HRESULT err;

// use a function to read the binary file into memory
pFnxVoice = (FnxVoicePtr) ReadBinaryFile( "<voice neural net
file>.vnn", &iSize );

// Initialize voice neural net.
err = FnxVoiceInit( pFnxVoice );
if (err)
    return err; // initialization error
```

Related topic

[Adding speech recognition](#)

FnxVoiceTiming

Description

Retrieves information on how long speech recognition is taking.

Syntax

```
int FnxVoiceTiming( FnxFnxVoiceUserPtr pFnxFnxVoiceUser, int *piFeatureTicks,
int *piNNetTicks, int *piSearchTicks, int *piTotalTicks, float *pfSecondsPerTick,
int *piNumFrames )
```

Parameters

<i>pFnxFnxVoiceUser</i>	FnxFnxVoiceUserPtr	Pointer to a voice user file initialized by FnxFnxVoiceUserInit . Results are returned for this user file or headset.
<i>piFeatureTicks</i>	int (optional)	Average number of CPU ticks per 10ms frame, spent in the feature extraction portion of recognition.
<i>piNNetTicks</i>	int (optional)	Average number of CPU ticks per 10ms frame, spent in the neural network portion of recognition.
<i>piSearchTicks</i>	int (optional)	Average number of CPU ticks per 10ms frame, spent in the search (decoding) portion of recognition.
<i>piTotalTicks</i>	int (optional)	Average number of CPU ticks per 10ms frame, spent on the entire speech recognition process after full processing begins.
<i>pfSecondsPerTick</i>	float (optional)	Number of seconds per CPU tick.
<i>piNumFrames</i>	int (optional)	Number of frames over which <i>piTotalTicks</i> was calculated.

Returns

FNX_ERR_NONE	Successful.
FNX_NULL_POINTER	<i>pFnxFnxVoiceUser</i> is NULL.
FNX_WRONG_TYPE	<i>pFnxFnxVoiceUser</i> does not point to a valid voice user file.
FNX_NOT_INITIALIZED	<i>pFnxFnxVoiceUser</i> is not initialized. See FnxFnxVoiceUserInit .

Comments

- Set any of the optional parameters to NULL if not needed.
- Feature extraction, but not full processing is performed on the initial frames of speech recognition (usually 11 frames). Because *piTotalTicks* is not computed on the faster initial frames, the average is not skewed smaller.
- 0 is returned to *piNNetTicks*, *piSearchTicks* and *piTotalTicks* during the first 11 frames of processing.

Example code

```
int iFeatureTicks, iNNetTicks, iSearchTicks, iTotalTicks;
float fSecondsPerTick;
HRESULT err;

err = FnxVoiceTiming( pCom->m_pVoiceUser, &iFeatureTicks,
&iNNetTicks, &iSearchTicks, &iTotalTicks, &fSecondsPerTick,
iNumFrames );
```

Related topic

[FnxVoiceUserInit](#)

FnxVoiceUserInit

Description

Initializes a voice user file (.usr). Call this API once for every user (connected communicator headset).

Syntax

HRESULT **FnxVoiceUserInit**(**FnxVoicePtr** *pFnxVoice*, **FnxVoiceUserPtr** *pFnxVoiceUser*)

Parameters

<i>pFnxVoice</i>	FnxVoicePtr	Pointer to voice neural net file initialized by FnxVoiceInit .
<i>pFnxVoiceUser</i>	FnxVoiceUserPtr	Pointer to a voice user file. The pointer is returned by ReadBinaryFile or similar function. See Example code below.

Returns

FNX_ERR_NONE	Successful.
FNX_NULL_POINTER	<i>pFnxVoice</i> or <i>pFnxVoiceUser</i> is NULL.
FNX_WRONG_TYPE	<i>pFnxVoice</i> does not point to a valid neural net file, or <i>pFnxVoiceUser</i> does not point to a valid user file.
FNX_INCOMPLETE	The footer on the <i>pFnxVoiceUser</i> block is incorrect (memory overrun).
FNX_VERSION_MISMATCH	<i>pFnxVoiceUser</i> is not the correct version (e.g., built by obsolete code).
FNX_NOT_INITIALIZED	<i>pFnxVoice</i> has not been initialized.
FNX_TAG_MISMATCH	<i>pFnxVoice</i> and <i>pFnxVoiceUser</i> were not created at the same time, and cannot be used together.
FNX_UNALIGNED_BLOCK	<i>pFnxVoice</i> or <i>pFnxVoiceUser</i> does not begin at an even 4-byte memory address. Rebuild vocabulary, voice neural net and user files.

Comments

Call **FnxVoiceInit** (returns *pFnxVoice* pointer) before calling **FnxVoiceUserInit**.

Example code

```
// Inside the initialization routine of a headset communicator.
FnxVoiceUserPtr pFnxVoiceUser;
```

```
// Use some function to read the binary file into memory
pFnxVoiceUser = (FnxVoiceUserPtr) ReadBinaryFile( "<voice user
file>.usr", &iSize );

// The pFnxVoice structure is passed in and is previously
initialized.
if( FnxVoiceUserInit( pFnxVoice, pFnxVoiceUser ) != 0 )
    return -1; // Error in initialization. Voice user file may not be
    valid.
```

Related topic

[Adding speech recognition](#)

FnxVoiceRecognize

Description

Performs speech recognition on a segment of speech data, and updates the internal state of *pFnxVoiceUser*.

Syntax

HRESULT **FnxVoiceRecognize**(**FnxVoicePtr** *pFnxVoice*, **FnxVoiceUserPtr** *pFnxVoiceUser*, **short** **pwWaveData*, **int** *iNumSamples*, **unsigned int** **puiResult*)

Parameters

<i>pFnxVoice</i>	FnxVoicePtr	Pointer to voice neural net file initialized by FnxVoiceInit .
<i>pFnxVoiceUser</i>	FnxVoiceUserPtr	Pointer to a voice user file initialized by FnxVoiceUserInit . Results are returned for this user file or headset.
<i>pwWaveData</i>	short	Array of speech samples on which to perform speech recognition.
<i>iNumSamples</i>	int	Number of speech samples in <i>pwWaveData</i> (e.g., 160).
<i>puiResult</i>	unsigned int	Address to receive one of the following flags: ▪ SPEECH_DETECTED: Beginning of speech was detected (this time or earlier). ▪ END_OF_SPEECH: End of speech has been detected (this time or earlier). ▪ RESULTS_AVAILABLE: Recognition has finished. Call FnxVoiceGetResults to retrieve the results.

Returns

FNX_ERR_NONE	Successful.
FNX_NULL_POINTER	<i>pFnxVoice</i> or <i>pFnxVoiceUser</i> is NULL.
FNX_WRONG_TYPE	<i>pFnxVoice</i> does not point to a valid voice

	neural net file, or <i>pFnxVoiceUser</i> does not point to a valid voice user file.
FNX_NOT_INITIALIZED	<i>pFnxVoice</i> or <i>pFnxVoiceUser</i> were not initialized by FnxVoiceInit or FnxVoiceUserInit .
FNX_SPEECH_ERROR	Unexpected error occurred during speech recognition.

Comments

- **FnxVoiceRecognize** updates the internal state of *pFnxVoiceUser* to reflect the recognition performed on the input speech samples.
- **Frame size.** Speech recognition is performed on 10 millisecond speech frames at a time, which is 160 samples at a sample rate of 16 kHz. **FnxVoiceRecognize** processes all speech frames. Passing in multiple frames takes longer than passing in a single frame. Left over fractions of a frame are stored internally until enough samples are passed to make a full frame.
- **Timing.** Because timing is so important in games, it is strongly recommended that the program pass 160 speech samples at a time (i.e., process exactly one frame at a time), or at least a multiple of 160 samples (fractional frames do not add up and cause additional processing at arbitrary times).

Recognition is performed in three steps: feature extraction, neural net calculation, and decoding. Feature extraction is performed on every 10 millisecond speech frame. Neural net calculations and decoding are performed after receiving approximately 11 frames of speech data, which provides enough context information. If doing timing tests, expect the first several frames to be extremely quick compared to the frames on which full processing is performed. See **FnxVoiceTiming**.

- **Speed.** Feature extraction takes a fixed amount of time (more or less). The neural net calculation is usually the slowest part of the process. If multiple neural network (.psi) files are available for a particular language, smaller files are usually faster. They may not be as accurate.

Decoding speed depends on the size of the vocabulary, or combined size of multiple vocabularies. It also depends on the pruning level set by **BuildVoice.exe** or **FnxBuildVoice**. The number of search paths reported by the tools for each vocabulary is proportional to the time spent in the decoding phase for that vocabulary when active. The tools enforce a minimum of 100 search paths per vocabulary to avoid large drops in accuracy.

Depending on the size of the vocabulary and the neural network, speech recognition can take from 5 to 15% of the CPU per user. Using 10% of the CPU requires 1 millisecond to process a 10 millisecond speech frame.

- **Multiple users.** Some games allow multiple users to talk at the same time using headset communicators. CPU requirements are approximately the same for each user. When the number of users exceeds CPU resources, the game can buffer the speech and process it as resources become available.

This causes a delayed response to the speech without slowing down the rest of the game. Game situations of this type are fairly uncommon. If a game event requires simultaneous voice input from multiple users, the game should devote the CPU towards speech recognition.

- 🔴 **Speech detection.** **FnxFVoiceRecognize** determines when speech starts (returns `SPEECH_DETECTED`). Actual speech may have started several frames earlier, since it takes a few frames to be sure that speech has started. All of the data is processed, however, and none is missed.

Recognition continues until 500 milliseconds of silence is detected (`END_OF_SPEECH` is returned). Any additional speech passed to **FnxFVoiceRecognize** is not processed. In this case, and until `RESULTS_AVAILABLE` is returned, call **FnxFVoiceRecognize** with *pwWaveData* set to `NULL`. Set *iNumSamples* to the size of the maximum number of frames of data to process before returning. For example, set *iNumSamples* to 160 to cause at most one frame to be flushed and processed.

When the recognition pipeline is empty, which may correspond to `END_OF_SPEECH`, `RESULTS_AVAILABLE` is returned. Call **FnxFVoiceGetResults** to retrieve recognition results.

Example code

```
// Inside the processing loop for a headset communicator.

FnxFVoicePtr pFnxFVoice; // Global voice neural net.
FnxFVoiceUserPtr pFnxFVoiceUser; // Global user or headset.
HRESULT err;

// An example of what state information the game can keep for each
// user:
enum {VOICE_IDLE, // push-to-talk button has not been pushed.
VOICE_LISTENING, // recognition is currently being done on mic input
VOICE_FINISHING, // END_OF_SPEECH detected, but not necessarily done
VOICE_DONE}; // RESULTS_AVAILABLE returned and results retrieved.

// Can return to VOICE_IDLE once the results are used.
DWORD uiResult;
int iState;

// For each microphone packet available...
if( iState == VOICE_LISTENING || iState == VOICE_FINISHING )
{
    // Submit the copied data to the speech recognizer
    if (iState == VOICE_FINISHING) // i.e., if we have already seen
        END_OF_SPEECH
    {
        // Provide time to finish doing recognition, but don't send
        // more data
        err = FnxFVoiceRecognize( pFnxFVoice, pVoiceUser, NULL, 0,
            &uiResult );
    }
    else // have not seen END_OF_SPEECH, so continue passing in real
        mic data
    {

```

```

        // Send in some wave data and do some processing (2 bytes per
        sample)
        err = FnxVoiceRecognize( pFnxVoice, pVoiceUser,
        pwMicrophoneData, iMicrophonePacketSizeInBytes / 2, &uiResult
        );

        if( uiResult & END_OF_SPEECH )
        {
            iState = VOICE_FINISHING;
        }
    }

    // Check to see if the recognition results are now available
    if( uiResult & RESULTS_AVAILABLE )
    {
        int iNBest;
        DWORD *pdwWordID, iBestWordID;
        FLOAT *pfConfidence, iBestConfidence;

        // Processing has completed on the speech, so get the results
        err = FnxVoiceGetResults(pFnxVoice, pVoiceUser, &pdwWordID,
        &pfConfidence, &iNBest);
        if( iNBest > 0 && !err )
        {
            // The top answer is in the first array element
            iBestWordID = pdwTempWordID[0];
            fBestConfidence = pfConfidence[0];
        }
    }
    iState = VOICE_DONE;
}

```

FnxVocabBld

Description

Creates a binary vocabulary file (.xvocab extension) from a word list.
VocabBld.exe also performs this task.

Syntax

HRESULT FnxVocabBld(**LPWSTR** *wsWordListFile*, **LPWSTR** *wsDictionaryFile*,
LPWSTR *wsNNetFile*, **LPWSTR** *wsXVocabFile*, **int** *iNBest*, **FLOAT** *fPruningLevel*,
LPWSTR *wsWordFile*)

Parameters

<i>wsWordListFile</i>	LPWSTR	Word list filename (.txt extension).
<i>wsDictionaryFile</i>	LPWSTR	Language dictionary file (.pdc extension). One is provided for each language (e.g. USEnglish.pdc).
<i>wsNNetFile</i>	LPWSTR	Language neural net file (.psi extension). One is provided for each language (e.g. USEnglish.psi).
<i>wsXVocabFile</i>	LPWSTR	Output vocabulary file (.xvocab extension).
<i>iNBest</i>	int	Number of top answers to find for <i>wsXVocabFile</i> . Default = 1.
<i>fPruningLevel</i>	FLOAT	This setting effects a trade-off between speed and recognition accuracy. Value range is 0 to 100: 0 is fastest; 50 is the default (fast and accurate); 100 is the most accurate.
<i>wsWordFile</i>	LPWSTR	Name of file created by FnxAdaptVocab.

Returns

FNX_ERR_NONE	Successful.
FNX_WORDLIST_READ_ERROR	Error reading <i>wsWordListFile</i> .
FNX_DICTIONARY_READ_ERROR	Error reading <i>wsDictionaryFile</i> , or a problem with a pronunciation exists.
FNX_PNI_READ_ERROR	Error reading <i>wsNNetFile</i> .
FNX_INIT_ERROR	Error initializing the recognizer.
FNX_WRITE_ERROR	Error writing <i>wsXVocabFile</i> .

Comments

- Word list format:

[ID] [tab] [word or phrase]\n

ID: positive integer from 0 to 65535 (returned for each recognized word).
Lines not beginning with a digit are ignored.

- Setting *iNBest* to a value greater than 1 increases memory and CPU requirements. Use the default (*iNBest* = 1) unless a game really needs to know what the 2nd to Nth best words or phrases are.
- For vocabularies of less than 10 words, pruning does not make a noticeable difference. For vocabularies greater than 100 words, set *fPruningLevel* to optimize the speed vs. accuracy trade-off. A value of 50 provides good recognition in most cases. Using a larger value may not increase accuracy. Using a smaller value can speed up recognition, but may also cause a decrease in accuracy.
- Many speech-enabled games use different vocabularies during the course of the game. Create a separate word file (*wsWordListFile*) for each vocabulary. Call **FnxVocabBld** for each word file.

Example code

```
int main( int argc, wchar_t *argv[ ])
{
    wchar_t *wsWordFile      = NULL;
    wchar_t *wsDictionaryFile = NULL;
    wchar_t *wsNNetFile      = NULL;
    wchar_t *wsXVocabFile    = NULL;
    int      iNBest=1;
                // N-best value to use for this vocabulary
                //(e.g., N=3 => always find the best three matching
                words)

    int      iPruneLevel = 50;
    int      iMainParam=1; // Which non-flagged parameter we're on
    (starting with #1)
    int      iArg; // Which argument in argv[] we're on
    HRESULT err;

    // Make sure we have enough command-line arguments.
    if( argc < 4 )
        PrintUsage(argv[0]);

    // Parse the command-line arguments.
    for( iArg = 1; iArg < argc; iArg++ )
    {
        if( argv[iArg][0] == '-' )
        {
            // Handle flags in whatever order they happen to appear.
            switch(argv[iArg][1])
            {
                case 'n': iNBest = _wtoi(argv[++iArg]); break;
                case 'p': iPruneLevel = _wtoi(argv[++iArg]); break;
            }
        }
    }
}
```

```

        // other arguments may go here if needed
        default: printf("Unknown argument '%s'\n", argv[iArg]);
        PrintUsage(argv[0]); // exits the program.
    }
}
else
{
    /* Handle non-flagged arguments in the order expected */
    // This depends on case strings 2,3,4 to be convertible to
    // multibyte strings.
    switch(iMainParam++)
    {
        case 1:
            wsWordFile      = argv[iArg];
            break;
        case 2:
            wsDictionaryFile = argv[iArg];
            break;
        case 3:
            wsNNetFile       = argv[iArg];
            break;
        case 4:
            wsXVocabFile     = argv[iArg];
            break;
        default: PrintUsage(argv[0]);
    }
}
}

if( iMainParam <= 4 || wsWordFile == NULL
    || wsDictionaryFile == NULL || wsNNetFile == NULL
    || wsXVocabFile == NULL )
{
    PrintUsage( argv[0] ); // exit if there is a parameter
    missing
}

/* Create the output file 'wsXVocabFile' and return the result
(0=>ok, -1=>error). */
err = FnxVocabBld( wsWordFile, wsDictionaryFile, wsNNetFile,
wsXVocabFile, iNBest, (FLOAT)iPruneLevel );

if( err )
{
    wprintf( L"Error in trying to create recognition vocabulary
'%s'.\n", wsXVocabFile );
    PrintUsage(argv[0]);
}
else wprintf( L"Wrote recognition vocabulary file '%s'.\n",
wsXVocabFile );
return 0;
} /* main */

```

FnxBuildVoice

Description

Creates a binary voice neural net file (.vnn extension) and a voice user file (.usr extension). **BuildVoice.exe** also performs this task.

Syntax

HRESULT FnxBuildVoice(**LPWSTR** *wsOutputFnxVoice*, **LPWSTR** *wsOutputFnxVoiceUser*, **LPWSTR** *wsNNetFile*, **LPWSTR** **pwsXVocabFiles*, **int** *iNumVocabFiles*, **int** *iMaxSimultaneousVocabs*, **int** *iMaxTotalSearchPaths*, **int** ***ppiNumSearchPaths*, **int** *iMaxTotalSearchNodes*, **int** ***ppiNumSearchNodes*)

Parameters

<i>wsOutputFnxVoice</i>	LPWSTR	Output voice neural net (.vnn) file.
<i>wsOutputFnxVoiceUser</i>	LPWSTR	Output voice user (.usr) file.
<i>wsNNetFile</i>	LPWSTR	Language neural net file (.psi extension). One is provided for each language (e.g. USEnglish.psi).
<i>pwsXVocabFiles</i>	LPWSTR	Array of <i>iNumVocabFiles</i> vocabulary files (.xvocab) use together with the two output files.
<i>iNumVocabFiles</i>	int	Number of vocabulary files in <i>pwsXVocabFiles</i> .
<i>iMaxSimultaneousVocabs</i>	int	Maximum number of vocabularies that can be selected simultaneously. See FnxSelectXVocabs and Comments below.
<i>iMaxTotalSearchPaths</i>	int	Maximum number of search paths expected at once (i.e., from using multiple vocabulary files). Default = 0.
<i>ppiNumSearchPaths</i>	int	Array in which to return the max search paths for each vocabulary. Default = 0.
<i>iMaxTotalSearchNodes</i>	int	Total number of search graph nodes allowed at one time.
<i>ppiNumSearchNodes</i>	int	Array in which to return the number of search nodes for each vocabulary.

Returns

FNX_ERR_NONE	Successful.
---------------------	-------------

FNX_PNI_READ_ERROR	Error reading <i>wsNNetFile</i> .
FNX_VOCAB_READ_ERROR	Error reading <i>pwsXVocabFiles</i> .
FNX_WRITE_ERROR	Error writing <i>wsOutputFnxVoice</i> and <i>wsOutputFnxVoiceUser</i> .

Comments

- Vocabulary files are rewritten to include the unique ID tag given to *wsOutputFnxVoice* and *wsOutputFnxVoiceUser*.
- All vocabularies may be selected at once. Providing (*iMaxSimultaneousVocabs*) the actual number of files to be selected at once saves memory. Many applications use only one vocabulary at a time. Others may make one vocabulary always available, plus another "current" vocabulary (set *iMaxSimultaneousVocabs* = 2). If is not know how many vocabularies will be used together, set *iMaxSimultaneousVocabs* to equal *iNumVocabFiles*. This ensures that enough space is allocated in the voice user file to handle any combination.
- Use search paths and search nodes to fine-tune memory. The number of vocabularies used simultaneously must be known. Adding up the maximum number of search paths and search nodes for each set of vocabularies provides the maximum possible sum of these values. CPU speed is not affected by these adjustments. If not needed, set *iMaxTotalSearchPaths* and *iMaxTotalSearchNodes* to NULL.

Example

A game has four vocabularies -- three vocabularies, only one of which will be active at once, and a global set of commands that can be used at any time. Assume that the number of search paths and search nodes reported by **BuildVoice.exe** are as follows:

Vocabulary	Search Paths	Search Nodes
Vocab1.xvocab	500	850
Vocab2.xvocab	700	1120
Vocab3.xvocab	100	130
Global.xvocab	150	300

The default is to allow all vocabularies at once, which results in allocating enough space to hold a total of $500+700+100+150=1450$ search paths and $850+1120+130+300=2400$ search nodes.

Setting *iMaxSimultaneousVocabs* = 2, space is allocated to hold the largest two vocabularies ($700+500=1200$ search paths and $1120+850=1970$ search nodes). Knowing, however, that the global vocabulary can only be joined with one other vocabulary at a time, results in the following
 $150+\max(500,700,100) = 150+700=850$ search paths and
 $300+\max(850,1120,130) = 300+1120=1420$ search nodes.

BuildVoice.exe displays search paths and search nodes to the console. **FnxBuildVoice** returns these values to *ppiNumSearchPaths* and *ppiNumSearchNodes* for each vocabulary. Use these values to calculate *iMaxTotalSearchPaths* and *iMaxTotalSearchNodes* if necessary. If not needed,

set each to NULL. Otherwise, the calling routine is responsible to free the returned arrays.

Another option is to set the output filenames to NULL and call **FnxBuildVoice** once to get the returned arrays. After calculating the values to use, call **FnxBuildVoice** again with the output filenames not set to NULL, and with *iMaxTotalSearchPaths* and *iMaxTotalSearchNodes* set to the calculated values.

Using the default values of 0 for *iMaxTotalSearchPaths* and *iMaxTotalSearchNodes* causes these values to be automatically computed from the other arguments.

Example code

```
int wmain(int argc, wchar_t *argv[])
{
    wchar_t *wsOutputVoice = NULL;
    wchar_t *wsVoiceUser = NULL;
    wchar_t *wsInputPNI = NULL;
    wchar_t **pwsXVocabFiles = NULL;
    int iMainParam=1; // Non-flagged param we're on (starting with
    #1)
    int iArg; // Which argument in argv[] we're on
    int iNumVocabFiles=0; // Number of vocabularies found so far.
    int iMaxVocabs=0; // 0=>use iNumXVocabs, all vocabs possible at
    once.
    int iMaxTotalSearchPaths=0; // 0=>use default, max of
    'iMaxVocabs' vocabs' counts.
    int iMaxTotalSearchNodes=0; // 0=>use default, max of
    'iMaxVocabs' vocabs' counts.
    int iTotalsearchNodes = 0;
    int *piNSPaths=NULL; // Maximum number of search paths for each
    vocabulary.
    int *piNSNodes=NULL; // Number of nodes for each vocabulary.
    int iTotalsearchPaths; // Total of all values in piNSPaths[].
    int iMaxPathIndex; // In pszXVocabFiles[], vocab w/ lgst search
    paths val.
    int iMaxNodeIndex; // In pszXVocabFiles[], vocab w/ lgst search
    nodes val.
    int iMaxIndex;
    int iVocab;
    HRESULT err;

    /* Make sure we have enough command-line arguments */
    if( argc < 4 )
        PrintUsage(argv[0]);

    /* Allocate an array that is at least large enough to hold all of
    the XVocab names */
    pwsXVocabFiles = (wchar_t **)calloc(argc-3, sizeof(wchar_t *));

    /* Parse the command-line arguments */
    for( iArg = 1; iArg < argc; iArg++ )
    {
        if (argv[iArg][0]=='-')
        {
```

```

/* Handle flags in whatever order they happen to come in
*/
switch(argv[iArg][1])
{
    case 's': iMaxVocabs = _wtoi(argv[++iArg]); break;
    case 'p': iMaxTotalSearchPaths = _wtoi(argv[++iArg]);
    break;
    case 'n': iMaxTotalSearchNodes = _wtoi(argv[++iArg]);
    break;

    // other arguments may go here if needed
    default: printf("Unknown argument '%s'\n", argv[iArg]);
    PrintUsage(argv[0]); // exits the program.
}
}
else
{
    /* Handle non-flagged arguments in the order expected */
    switch(iMainParam++)
    {
        case 1: wsOutputVoice = argv[iArg]; break;
        case 2: wsVoiceUser = argv[iArg]; break;
        case 3: wsInputPNI = argv[iArg]; break;
        default: pwsXVocabFiles[iNumVocabFiles++] = argv[iArg];
        break;
    }
}
} /* for each argument */

/* Make sure we got the required arguments */
if( iMainParam <= 3 || wsOutputVoice==NULL || wsInputPNI==NULL ||
iNumVocabFiles < 1 )
{
    printf("Missing arguments.\n");
    PrintUsage(argv[0]); // exit if there is a parameter missing
}

/* If not overridden, use the default of allowing all
vocabularies at once */
if( iMaxVocabs <= 0 )
    iMaxVocabs = iNumVocabFiles;

/* Create output file, & allo. storage in piNSPaths[] and
piNSNodes[]. */
err = FnxBuildVoice( wsOutputVoice, wsVoiceUser, wsInputPNI,
    pwsXVocabFiles, iNumVocabFiles, iMaxVocabs,
    iMaxTotalSearchPaths, &piNSPaths,
    iMaxTotalSearchNodes, &piNSNodes );

if (err)
{
    wprintf(L"Error in trying to create binary nnet file '%s'\n",
    wsOutputVoice);
    PrintUsage(argv[0]);
}
}

```

```

/* Display the vocabularies and number of search paths for each
*/
printf("Search Paths  Search Nodes Vocab File\n");
iTotalSearchPaths = 0;
iMaxIndex = iMaxTotalSearchPaths = 0;

for (iVocab=0;iVocab < iNumVocabFiles; iVocab++)
{
    wprintf(L"%12d  %12d  %s\n", piNSPaths[iVocab],
        piNSNodes[iVocab], pwsXVFles[iVocab]);
    iTotalSearchPaths += piNSPaths[iVocab];

    if (piNSPaths[iVocab]>iMaxTotalSearchPaths)
    {
        iMaxTotalSearchPaths = piNSPaths[iVocab];
        iMaxPathIndex = iVocab;
    }

    iTotalSearchNodes += piNSNodes[iVocab];
    if (piNSNodes[iVocab]>iMaxTotalSearchNodes)
    {
        iMaxTotalSearchNodes = piNSNodes[iVocab];
        iMaxNodeIndex = iVocab;
    }
}

if (iNumVocabFiles>1)
{
    wprintf(L"%6d total paths in all %d vocabularies; ",
        iTotalSearchPaths, iNumVocabFiles);
    wprintf(L"%6d paths in largest vocabulary %s\n",
        iMaxTotalSearchPaths, pwsXVocabFiles[iMaxPathIndex]);
    wprintf(L"%6d total nodes in all %d vocabularies; ",
        iTotalSearchNodes, iNumVocabFiles);
    wprintf(L"%6d nodes in largest vocabulary %s\n",
        iMaxTotalSearchNodes, pwsXVocabFiles[iMaxNodeIndex]);
}

wprintf(L"%12d  total paths maximum allowed at once in %s\n",
    piNSPaths[iNumVocabFiles], wsOutputVoice);

/* Free up memory allocated within FnxBuildVoice() */
free(piNSPaths);
free(piNSNodes);
return 0;
} /* main */

```